FUNDAMENTOS Y RELACIONES

La construcción de software y sus áreas de apoyo

Objetivo. Comprender la construcción de software y comparar su relación con el análisis de requisitos, el diseño, las pruebas y el mantenimiento mediante un cuadro, un esquema y argumentos fundamentados.

La construcción convierte decisiones y necesidades en software ejecutable mediante codificación, verificación, integración y depuración. También incluye pruebas unitarias y de integración; por ello, sus límites se superponen con el diseño y las pruebas (Washizaki, 2026, cap. 4).

1. Cuadro comparativo

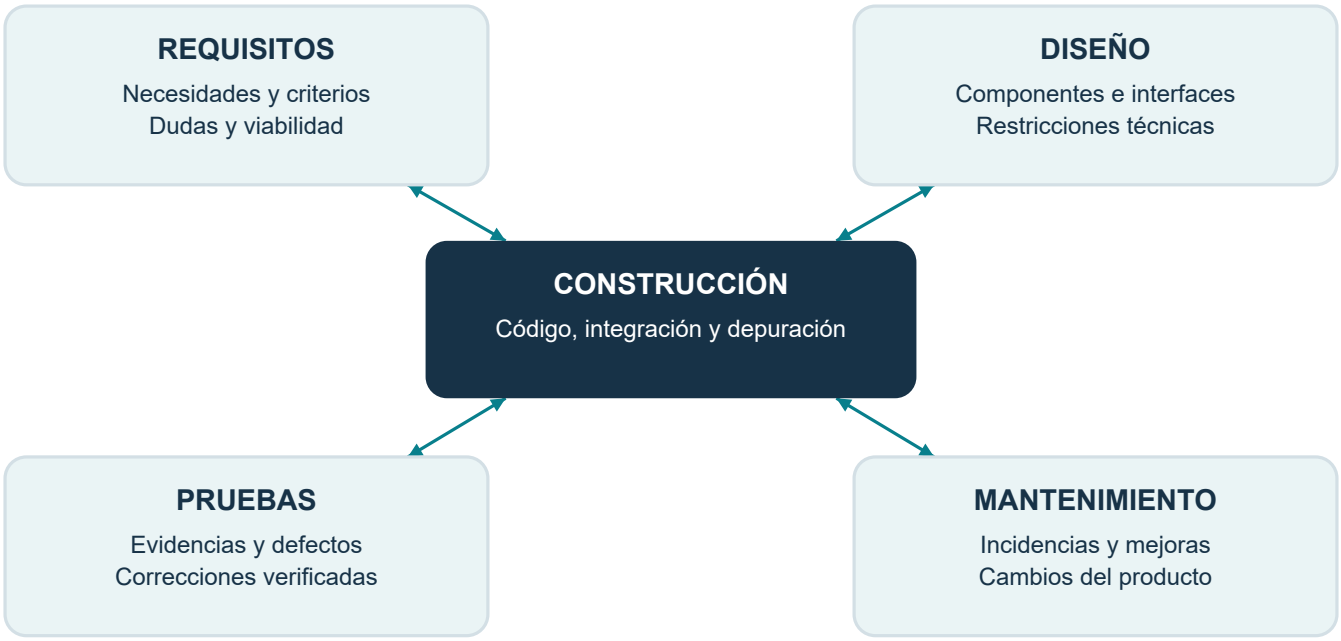
Área	Definición	Propósito	Relación con la construcción
Análisis de requisitos	Examina las necesidades, sus implicaciones y posibles conflictos. Precisa funciones, restricciones y atributos de calidad.	Acordar qué problema resolver y establecer condiciones claras y verificables.	Orienta lo que se programa y sus criterios de aceptación. Las dudas técnicas del desarrollo permiten ajustar los requisitos.
Diseño de software	Transforma requisitos en especificaciones de componentes, interfaces, datos y comportamiento.	Definir una solución implementable y comprensible.	Guía el código; los hallazgos al implementarlo pueden exigir revisar el diseño.
Pruebas de software	Actividades estáticas y dinámicas que evalúan productos de trabajo y software para descubrir defectos y comprobar requisitos.	Aportar evidencia de calidad y reducir el riesgo de fallos.	Los desarrolladores prueban unidades e integración. Los defectos detectados motivan correcciones y nuevas verificaciones.
Mantenimiento de software	Modificación del software existente para corregir defectos, adaptarlo o mejorarlo.	Conservar su utilidad y calidad ante nuevas necesidades.	Los cambios requieren analizar, diseñar, programar y volver a probar; el código comprensible facilita esta labor.

Nota. Elaboración propia. Requisitos: Jonker y Gomstyn (s. f.) y Washizaki (2026, cap. 1). Diseño y mantenimiento: Washizaki (2026, caps. 3 y 7). Pruebas: International Software Testing Qualifications Board [ISTQB] (2024).

Lectura comparativa. Requisitos define qué se necesita; diseño, cómo organizar la solución; pruebas, qué evidencia respalda su calidad; y mantenimiento, cómo conservarla y hacerla evolucionar.

2. Una relación de retroalimentación

Figura 1
Intercambios entre la construcción y las áreas estudiadas



Nota. Esquema de elaboración propia. Las flechas dobles representan intercambio de información, no una secuencia rígida. Las áreas pueden colaborar dentro de una misma iteración (Washizaki, 2026, cap. 4).

Caso práctico: reservas de un laboratorio

Se propone un sistema universitario para reservar equipos. El siguiente ejemplo es hipotético y muestra cómo una misma regla necesita decisiones compartidas:

Requisitos. Se acuerda la regla R-01: un equipo no admite dos reservas activas en el mismo horario. El criterio de aceptación exige que, ante dos solicitudes simultáneas, solo una se confirme y la otra reciba un aviso.

Diseño y construcción. Se separan la interfaz, la lógica de reservas y el acceso a datos. El equipo desarrolla una operación transaccional que comprueba y registra la disponibilidad de forma indivisible; una consulta previa desde la pantalla sería insuficiente.

Pruebas. Se ejecutan solicitudes concurrentes y se comprueba que solo quede una reserva activa. También se prueban horarios consecutivos, cancelaciones y reintentos. Si aparecen duplicados, el defecto vuelve a construcción y puede obligar a revisar el diseño.

Mantenimiento. Si luego se permiten reservas por intervalos variables, se analiza el impacto sobre R-01, se adapta la detección de solapamientos y se actualizan las pruebas. Corregir únicamente la pantalla dejaría inconsistente el sistema.

Trazabilidad del ejemplo: R-01 → decisión de diseño → operación de reserva → prueba de concurrencia → cambio controlado. El vínculo permite localizar qué debe revisarse cuando cambia la regla.

3. Análisis de la pregunta guía

¿Por qué la construcción de software no puede realizarse de forma aislada y depende del trabajo colaborativo con otras áreas?

Porque producir código ejecutable exige decisiones sobre necesidades, estructura, calidad y evolución. Una sola persona puede asumir varias funciones, pero necesita integrar las perspectivas de esas áreas y contrastar sus decisiones con usuarios y colaboradores.

Argumento 1. Programar bien exige acordar qué significa resolver el problema

Considero que la destreza técnica no reemplaza el conocimiento del contexto. En el caso del laboratorio, un programador podría interpretar “equipo disponible” como “equipo habilitado”, mientras el usuario entiende “equipo libre en ese horario”. El código podría funcionar según esa interpretación y aun así entregar una solución equivocada. Mi argumento es que la colaboración convierte supuestos individuales en acuerdos verificables: usuarios y analistas precisan la regla; desarrolladores señalan límites técnicos; y pruebas propone situaciones que revelan ambigüedades. La trazabilidad de requisitos ayuda a mantener esos acuerdos durante los cambios (Jonker & Gomstyn, s. f.).

Argumento 2. La calidad necesita evidencias y perspectivas complementarias

A mi juicio, quien construye una función puede concentrarse en el recorrido que espera que siga el usuario y pasar por alto combinaciones imprevistas. En las reservas, comprobar que cada solicitud funciona por separado no demuestra que dos solicitudes simultáneas sean seguras. Por eso propongo revisar los criterios y diseñar escenarios de error mientras se desarrolla. El diseño aporta mecanismos de consistencia; construcción los implementa; y pruebas comprueba sus resultados. El ISTQB (2024) señala que probar no demuestra la ausencia de defectos. La colaboración reduce puntos ciegos y aporta razones para aceptar el producto, sin prometer una seguridad absoluta.

Argumento 3. Cada decisión de construcción condiciona los cambios futuros

Sostengo que una entrega debe valorarse también por la facilidad para modificarla. Si la regla de reservas se repite en varias pantallas, un cambio posterior puede corregir unas y dejar otras inconsistentes. Consultar con diseño y mantenimiento permite concentrar esa regla, documentar su intención y conservar pruebas de regresión. Fowler (2019) explica que las deficiencias internas generan esfuerzo adicional al introducir cambios. De ello concluyo que conviene priorizar mejoras en los componentes que cambian con frecuencia, en lugar de acumular parches. Esta coordinación requiere tiempo, pero protege la continuidad del servicio y evita trasladar decisiones sin explicar a quienes mantendrán el sistema.

Conclusión

La construcción materializa la solución, pero su valor depende de acuerdos y evidencias compartidos. En el ejemplo, la regla de disponibilidad conecta las cuatro áreas con el código. Colaborar permite detectar errores de interpretación, verificar comportamientos críticos y modificar el producto de manera coherente durante su vida útil.

Referencias bibliográficas

Formato APA, séptima edición

Fowler, M. (2019, 21 de mayo). *Technical debt.* Martin Fowler. https://martinfowler.com/bliki/TechnicalDebt.html

International Software Testing Qualifications Board. (2024). *Certified tester foundation level syllabus* (Versión 4.0.1). https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Jonker, A., & Gomstyn, A. (s. f.). *What is requirements management?* IBM. https://www.ibm.com/think/topics/what-is-requirements-management

Washizaki, H. (Ed.). (2026). *Guide to the software engineering body of knowledge* (Versión 4.0a). IEEE Computer Society. https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf